

Portafolio

Checklist de migración: Portafolio HTML → Portafolio Espacial React + Vite

Sigue las fases en orden. Cada ítem tiene una casilla ☐ para marcar tu progreso. Los bloques de código son los cambios concretos a realizar en cada archivo.

FASE 0 — Preparación del entorno

0.1 Herramientas necesarias

- ☐ Node.js instalado (v18 o superior) — verificar con `node -v`
- ☐ npm instalado — verificar con `npm -v`
- ☐ Git instalado y configurado
- ☐ Editor con extensión **ES7+ React/Redux/React-Native snippets** (VSCode)
- ☐ Extensión **Prettier** configurada para formatear JSX

0.2 Respalidar el proyecto actual

- ☐ Hacer commit de todo tu portafolio HTML actual
 - ☐ Crear una rama de respaldo: `git checkout -b backup/portfolio-html`
 - ☐ Regresar a main: `git checkout main`
 - ☐ (Opcional) Subir backup a GitHub como tag: `git tag v1.0-html-portfolio`
-

FASE 1 — Crear el proyecto React + Vite

1.1 Inicialización

- ☐ Ejecutar en la carpeta donde vivirá el nuevo proyecto:

bash

```
npm create vite@latest portfolio -- --template react
cd portfolio
npm install
```

- ☐ Verificar que `npm run dev` levanta el servidor en `localhost:5173`
- ☐ Eliminar los archivos de ejemplo que Vite genera:
 - ☐ Borrar `src/App.css`
 - ☐ Borrar `src/assets/react.svg`
 - ☐ Limpiar el contenido de `src/App.jsx` (dejar solo la función vacía)
 - ☐ Limpiar el contenido de `src/index.css`

1.2 Instalar todas las dependencias del proyecto

Las librerías se instalan de una vez. Cada grupo tiene su propósito:

bash

```
# 3D y escena espacial
npm install @react-three/fiber @react-three/drei three @react-three/postprocessing

# Animaciones de UI y scroll
npm install gsap @gsap/react

# Partículas 2D de respaldo y scroll suave
npm install @tsparticles/react @tsparticles/slim lenis

# Internacionalización (reemplaza tu sistema lang/ + lenguaje.js)
npm install i18next react-i18next i18next-browser-languagedetector

# Enrutamiento (reemplaza tus múltiples .html)
npm install react-router-dom

# Iconos (reemplaza bootstrap-icons)
npm install lucide-react
```

1.3 Crear la estructura de carpetas de `src/`

- ☐ Crear las carpetas manualmente o con este comando desde la raíz del proyecto:

bash

```
mkdir -p src/components src/sections src/hooks src/assets/images
src/assets/icons src/styles src/locales/en src/locales/es src/data src/3d
```

La estructura final de `src/` que debes tener:

```
src/
├─ main.jsx
├─ App.jsx
├─ components/      ← Piezas de UI reutilizables
├─ sections/        ← Secciones únicas del portfolio
├─ hooks/           ← Lógica desacoplada de la UI
├─ assets/
│   ├─ images/      ← Tus imágenes actuales migradas aquí
│   └─ icons/       ← Tus SVGs actuales migrados aquí
├─ styles/          ← CSS global y variables
├─ locales/         ← Textos en ES e EN (reemplaza carpeta lang/)
│   ├─ en/
│   └─ es/
├─ data/            ← Arrays de proyectos, skills, etc.
└─ 3d/              ← Componentes exclusivos de Three.js
```

FASE 2 — Migrar archivos estáticos

2.1 Carpeta `images/` → `src/assets/images/`

- ☐ Copiar `images/yo.png` → `src/assets/images/yo.png`
- ☐ Copiar `images/projects/comercializadora.png` → `src/assets/images/projects/`
- ☐ Copiar `images/projects/panaderia.png` → `src/assets/images/projects/`
- ☐ Copiar `images/projects/portafolio-alejandro.png` → `src/assets/images/projects/`
- ☐ Copiar `images/projects/siki-mexico.png` → `src/assets/images/projects/`
- ☐ Copiar `images/projects/sistemas.png` → `src/assets/images/projects/`
- ☐ Copiar `images/redes-sociales/linkedin.png` → `src/assets/images/socials/`
- ☐ `images/mundo.gif` → Ya no se necesita. La animación del mundo se reemplaza por la escena 3D de Three.js.
- ☐ `images/contactme.png`, `images/project.jpg`, `images/services.jpg` → Evaluar si aún las usas; si sí, copiarlas a `src/assets/images/`.
- ☐ `videos/` → Mover a `public/videos/` (los vídeos se sirven estáticamente, no se importan en JS).

2.2 Carpeta `icons/` → `src/assets/icons/`

- ☐ Copiar todos los SVGs a `src/assets/icons/`
- ☐ **Nota importante:** Con `lucide-react` muchos de estos SVGs quedan obsoletos. Al llegar a cada sección verás qué ícono de Lucide los sustituye. Los SVGs propios y muy

específicos (como los de skills) sí se conservan.

2.3 Carpeta `fonts/` → eliminada

- ☐ Bootstrap Icons como fuente ya **no se usa**. Se instala `lucide-react` como librería React.
- ☐ Si tienes fuentes tipográficas propias en `fonts/`, copiarlas a `src/assets/fonts/`.
- ☐ Las fuentes del portfolio espacial (ej. Rajdhani, Space Grotesk) se cargarán desde Google Fonts en `index.html`.

FASE 3 — Configuración global del proyecto

3.1 `index.html` (raíz del proyecto, no en `src/`)

En Vite, `index.html` vive en la raíz. Reemplaza el contenido completo:

html

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="Portfolio de Alejandro – Desarrollador Web" />

    <!-- Fuentes espaciales desde Google Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?family=Rajdhani:wght@300;400;600;700&family=Inter:wght@300;400;500&display=swap" rel="stylesheet" />

    <title>Alejandro – Desarrollador Web</title>
  </head>
  <body>
    <!-- React monta aquí. No hay más HTML manual -->
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Qué cambia respecto a tu `index.html` actual:

- Ya no hay links a Bootstrap, jQuery ni tus CSS individuales — todo entra por React
- No hay scripts al final del body — Vite los gestiona
- El `<div id="root">` es el único punto de montaje

3.2 `src/main.jsx` — punto de entrada de React

jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './styles/globals.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
)
```

3.3 `src/styles/globals.css` — reemplaza todos tus CSS actuales

CSS

```
/* Variables de la paleta espacial */
:root {
  --color-bg: #050816; /* Negro espacial profundo */
  --color-bg-card: #0d1224; /* Azul oscuro para cards */
  --color-accent: #6366f1; /* Violeta estelar principal */
  --color-accent-2: #22d3ee; /* Cian nebulosa secundario */
  --color-text: #e2e8f0; /* Blanco suave */
  --color-muted: #64748b; /* Gris para texto secundario */
  --font-display: 'Rajdhani', sans-serif;
  --font-body: 'Inter', sans-serif;
}

/* Reset base */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

html { scroll-behavior: smooth; }

body {
```

```

background-color: var(--color-bg);
color: var(--color-text);
font-family: var(--font-body);
font-size: 16px;
line-height: 1.6;
overflow-x: hidden;
}

/* Tipografía base */
h1, h2, h3, h4 {
  font-family: var(--font-display);
  font-weight: 700;
  line-height: 1.2;
}

a { color: var(--color-accent); text-decoration: none; }
a:hover { color: var(--color-accent-2); }

img { max-width: 100%; display: block; }

/* Clase utilitaria para el canvas 3D */
.canvas-wrapper {
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  z-index: 0;
  pointer-events: none;
}

/* Contenido sobre el canvas */
.content-layer {
  position: relative;
  z-index: 1;
}

```

Qué reemplaza: `templatemo-first-portfolio-style.css`, `bootstrap-icons.css`, `magnific-popup.css`, `slide.css`, `nube/nube.css` y los estilos inline que tengas. Los CSS específicos de componente (`proyectos.css`) se convierten en CSS Modules dentro de su respectivo componente.

FASE 4 — Internacionalización: reemplazar `lang/` + `lenguaje.js`

Tu sistema actual carga JSONs y swappea texto con JS. En React usas **i18next**, que hace esto de forma declarativa.

4.1 Crear `src/locales/es/translation.json`

json

```
{
  "nav": {
    "home": "Inicio",
    "about": "Sobre mí",
    "projects": "Proyectos",
    "skills": "Habilidades",
    "contact": "Contacto"
  },
  "hero": {
    "greeting": "Hola, soy",
    "name": "Alejandro",
    "role": "Desarrollador Web Full Stack",
    "cta": "Ver mis proyectos",
    "ctaSecondary": "Descargar CV"
  },
  "about": {
    "title": "Sobre mí",
    "description": "Desarrollador apasionado por crear experiencias web únicas...",
    "yearsExp": "años de experiencia",
    "projectsDone": "proyectos completados"
  },
  "projects": {
    "title": "Mis proyectos",
    "viewCode": "Ver código",
    "viewDemo": "Ver demo",
    "close": "Cerrar"
  },
  "skills": {
    "title": "Habilidades",
    "frontend": "Frontend",
    "backend": "Backend",
    "tools": "Herramientas"
  },
  "contact": {
    "title": "Contáctame",
    "name": "Nombre",
    "email": "Correo electrónico",
    "message": "Mensaje",
  }
}
```

```
    "send": "Enviar mensaje",
    "success": "¡Mensaje enviado con éxito!",
    "error": "Hubo un error. Intenta de nuevo."
  }
}
```

4.2 Crear `src/locales/en/translation.json`

json

```
{
  "nav": {
    "home": "Home",
    "about": "About me",
    "projects": "Projects",
    "skills": "Skills",
    "contact": "Contact"
  },
  "hero": {
    "greeting": "Hi, I'm",
    "name": "Alejandro",
    "role": "Full Stack Web Developer",
    "cta": "View my projects",
    "ctaSecondary": "Download CV"
  },
  "about": {
    "title": "About me",
    "description": "Developer passionate about creating unique web experiences...",
    "yearsExp": "years of experience",
    "projectsDone": "completed projects"
  },
  "projects": {
    "title": "My projects",
    "viewCode": "View code",
    "viewDemo": "View demo",
    "close": "Close"
  },
  "skills": {
    "title": "Skills",
    "frontend": "Frontend",
    "backend": "Backend",
    "tools": "Tools"
  },
  "contact": {
```



```

    "title": "Contact me",
    "name": "Name",
    "email": "Email address",
    "message": "Message",
    "send": "Send message",
    "success": "Message sent successfully!",
    "error": "Something went wrong. Please try again."
  }
}

```

4.3 Crear `src/locales/index.js` — configuración de `i18next`

js

```

// Reemplaza: js/lenguaje.js, lang/en.json, lang/es.json
import i18n from 'i18next'
import { initReactI18next } from 'react-i18next'
import LanguageDetector from 'i18next-browser-languagedetector'
import translationES from './es/translation.json'
import translationEN from './en/translation.json'

i18n
  .use(LanguageDetector) // Detecta el idioma del navegador
  .automaticly
  .use(initReactI18next)
  .init({
    resources: {
      es: { translation: translationES },
      en: { translation: translationEN }
    },
    fallbackLng: 'es', // Si no detecta idioma, usa español
    interpolation: { escapeValue: false }
  })

export default i18n

```

Cómo se usa en cualquier componente:

jsx

```

import { useTranslation } from 'react-i18next'

function Hero() {
  const { t, i18n } = useTranslation()

```

```

    return (
      <h1>{t('hero.greeting')} {t('hero.name')}</h1>
    )
  }
}

```

Selector de idioma (reemplaza tu botón actual):

jsx

```

function LangToggle() {
  const { i18n } = useTranslation()
  const toggleLang = () => i18n.changeLanguage(i18n.language === 'es' ? 'en' : 'es')

  return (
    <button onClick={toggleLang}>
      {i18n.language === 'es' ? 'EN' : 'ES'}
    </button>
  )
}

```

4.4 Datos de proyectos multiidioma

Tu lang/projects-es.json y lang/projects-en.json se convierten en src/data/projects.js:

js

```

// src/data/projects.js
// Reemplaza: lang/projects-es.json y lang/projects-en.json
export const projects = [
  {
    id: 1,
    slug: 'comercializadora',
    image: '/src/assets/images/projects/comercializadora.png',
    github: 'https://github.com/tu-usuario/comercializadora',
    demo: '',
    techs: ['HTML', 'CSS', 'JavaScript'],
    // Textos en ambos idiomas dentro del mismo objeto
    es: {
      title: 'Comercializadora',
      description: 'Sistema de gestión para empresa comercializadora...'
    },
    en: {
      title: 'Commerce System',

```

```

      description: 'Management system for a commercial company...'
    }
  },
  {
    id: 2,
    slug: 'panaderia',
    image: '/src/assets/images/projects/panaderia.png',
    github: '',
    demo: '',
    techs: ['React', 'Node.js', 'MongoDB'],
    es: { title: 'Panadería', description: '...' },
    en: { title: 'Bakery App', description: '...' }
  }
  // ... resto de tus proyectos
]

```

FASE 5 — Migrar los archivos JS → componentes React y hooks

5.1 js/404.js → src/sections/NotFound.jsx

Tu `404.js` actual probablemente redirige o muestra un mensaje. En React es una ruta:

jsx

```

// src/sections/NotFound.jsx
// Reemplaza: 404.html + js/404.js
import { useTranslation } from 'react-i18next'
import { Link } from 'react-router-dom'

function NotFound() {
  const { t } = useTranslation()
  return (
    <div className="not-found">
      <h1>404</h1>
      <p>Esta página no existe en la galaxia.</p>
      <Link to="/">Volver al inicio</Link>
    </div>
  )
}

export default NotFound

```

En `App.jsx` se registra así:

jsx

```
import { Routes, Route } from 'react-router-dom'
import NotFound from './sections/NotFound'

<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

5.2 `js/alert-toast.js` → hook `src/hooks/useToast.js`

js

```
// src/hooks/useToast.js
// Reemplaza: js/alert-toast.js
import { useState, useCallback } from 'react'

export function useToast() {
  const [toast, setToast] = useState({ visible: false, message: '', type: 'success' })

  const showToast = useCallback((message, type = 'success') => {
    setToast({ visible: true, message, type })
    setTimeout(() => setToast(t => ({ ...t, visible: false })), 3500)
  }, [])

  return { toast, showToast }
}
```

Cómo se usa en el componente de contacto:

jsx

```
const { toast, showToast } = useToast()
// Al enviar el form exitosamente:
showToast(t('contact.success'), 'success')
// Al fallar:
showToast(t('contact.error'), 'error')
```

5.3 `js/click-scroll.js` → comportamiento nativo de React Router

Tu `click-scroll.js` hace scroll suave al hacer clic en los links del nav. Esto se resuelve con Lenis (scroll suave global) + anclas de sección:

jsx

```
// En Navbar.jsx, los links son así:
<a href="#proyectos" onClick={(e) => {
  e.preventDefault()
  document.getElementById('proyectos').scrollIntoView({ behavior: 'smooth'
})
}}>
  {t('nav.projects')}
</a>
```

Con Lenis instalado, el scroll suave es automático para toda la página. El archivo `click-scroll.js` queda eliminado.

5.4 js/custom.js → lógica distribuida por componentes

`custom.js` en portfolios plantilla suele tener lógica mezclada (navbar sticky, animaciones, etc.). Sepárala así:

- **Navbar sticky** → `src/hooks/useScrollPosition.js`

js

```
// Reemplaza la parte de sticky navbar de custom.js
import { useState, useEffect } from 'react'

export function useScrollPosition() {
  const [scrollY, setScrollY] = useState(0)

  useEffect(() => {
    const handler = () => setScrollY(window.scrollY)
    window.addEventListener('scroll', handler, { passive: true })
    return () => window.removeEventListener('scroll', handler)
  }, [])

  return scrollY
}
```

En `Navbar.jsx`:

jsx

```
const scrollY = useScrollPosition()
const isScrolled = scrollY > 50
// Aplica clase CSS si está scrolleado:
<nav className={`navbar ${isScrolled ? 'navbar--scrolled' : ''}`}>
```

- **Animaciones de entrada** → GSAP en cada componente (ver Fase 7)
- **Cualquier otra lógica de custom.js** → evalúa qué hacía y crea el hook o componente correspondiente

5.5 js/jquery.magnific-popup.min.js + js/magnific-popup-options.js + css/magnific-popup.css → modal React nativo

Tu galería de proyectos con popup se reemplaza por un componente modal propio, sin jQuery:

jsx

```
// src/components/ProjectModal.jsx
// Reemplaza: jquery.magnific-popup.min.js + magnific-popup-options.js +
css/magnific-popup.css
import { useEffect } from 'react'
import { useTranslation } from 'react-i18next'
import { X } from 'lucide-react'

function ProjectModal({ project, onClose }) {
  const { t, i18n } = useTranslation()
  const lang = i18n.language

  // Cerrar con Escape
  useEffect(() => {
    const handler = (e) => e.key === 'Escape' && onClose()
    window.addEventListener('keydown', handler)
    return () => window.removeEventListener('keydown', handler)
  }, [onClose])

  if (!project) return null

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={(e) => e.stopPropagation()}>
        <button className="modal-close" onClick={onClose} aria-
label="Cerrar">
          <X size={20} />
        </button>
        <img src={project.image} alt={project[lang].title} />
      </div>
    </div>
  )
}
```

```

    <h3>{project[lang].title}</h3>
    <p>{project[lang].description}</p>
    <div className="modal-techs">
      {project.techs.map(tech => <span key={tech}>{tech}</span>)}
    </div>
    <div className="modal-actions">
      {project.github && <a href={project.github} target="_blank">
        {t('projects.viewCode')}</a>}
      {project.demo && <a href={project.demo} target="_blank">
        {t('projects.viewDemo')}</a>}
    </div>
  </div>
</div>
)
}

export default ProjectModal

```

5.6 js/jquery.min.js + js/jquery.sticky.js → eliminados

jQuery ya no se usa. Todo lo que hacía jQuery lo hace React de forma nativa. El sticky del navbar lo maneja `useScrollPosition()`.

5.7 js/lenguaje.js → eliminado

Reemplazado completamente por `src/locales/index.js` + i18next (ver Fase 4).

5.8 js/multilingual-project-modal.js → integrado en ProjectModal.jsx

La lógica de cambiar el texto del modal según el idioma ya está en `ProjectModal.jsx` con `project[lang].title`. Archivo eliminado.

5.9 js/particles.js → src/3d/SpaceBackground.jsx

Tu `particles.js` actual genera partículas. Se reemplaza por la escena 3D de Three.js que es mucho más potente:

jsx

```

// src/3d/SpaceBackground.jsx
// Reemplaza: js/particles.js
import { Canvas } from '@react-three/fiber'
import { Stars, Float } from '@react-three/drei'
import { EffectComposer, Bloom } from '@react-three/postprocessing'

```

```

function SpaceBackground() {
  return (
    <div className="canvas-wrapper">
      <Canvas camera={{ position: [0, 0, 1] }}>
        {/* Campo de estrellas 3D – reemplaza tus partículas 2D */}
        <Stars
          radius={100}      // Radio de la esfera de estrellas
          depth={50}        // Profundidad de dispersión
          count={5000}      // Número de estrellas
          factor={4}        // Tamaño de las estrellas
          saturation={0.5}  // Saturación de color
          fade              // Fade en los bordes
          speed={0.5}       // Velocidad de rotación
        />

        {/* Efecto de bloom: hace brillar las estrellas */}
        <EffectComposer>
          <Bloom
            intensity={0.4}
            luminanceThreshold={0.3}
            luminanceSmoothing={0.9}
          />
        </EffectComposer>
      </Canvas>
    </div>
  )
}

export default SpaceBackground

```

5.10 js/progress-bar.js → src/components/SkillBar.jsx

jsx

```

// src/components/SkillBar.jsx
// Reemplaza: js/progress-bar.js
import { useEffect, useRef, useState } from 'react'

function SkillBar({ name, percentage, icon }) {
  const [animated, setAnimated] = useState(false)
  const ref = useRef(null)

  // Anima la barra cuando entra en viewport (reemplaza tu lógica de scroll
  // en progress-bar.js)

```



```

useEffect(() => {
  const observer = new IntersectionObserver(
    ([entry]) => { if (entry.isIntersecting) setAnimated(true) },
    { threshold: 0.3 }
  )
  if (ref.current) observer.observe(ref.current)
  return () => observer.disconnect()
}, [])

return (
  <div ref={ref} className="skill-bar">
    <div className="skill-bar__header">
      {icon} && <img src={icon} alt={name} width={24} height={24} />
      <span>{name}</span>
      <span>{percentage}%</span>
    </div>
    <div className="skill-bar__track">
      <div
        className="skill-bar__fill"
        style={{
          width: animated ? `${percentage}%` : '0%',
          transition: 'width 1.2s cubic-bezier(0.4, 0, 0.2, 1)'
        }}
      />
    </div>
  </div>
)
}

export default SkillBar

```

5.11 js/services.js → src/data/services.js + src/sections/Services.jsx

Tu `services.js` probablemente contiene los datos de servicios. Separa datos de UI:

js

```

// src/data/services.js – solo los datos
export const services = [
  {
    id: 'web-app',
    iconFile: 'app-web.svg', // Mantiene referencia a tu SVG existente
    es: { title: 'Aplicaciones Web', description: 'Desarrollo de apps web a medida...' },
  },

```

```

    en: { title: 'Web Applications', description: 'Custom web app
development...' }
  },
  // ... resto de servicios
]

```

5.12 js/slide.js + css/slide.css → eliminados

El slider/carrusel se reemplaza por el grid animado de proyectos con GSAP. No hay carrusel en el nuevo diseño espacial — los proyectos se muestran en grid con cards interactivas.

FASE 6 — Migrar las vistas HTML → componentes de sección

6.1 index.html → src/sections/Hero.jsx

El hero es la sección más importante visualmente. Estructura base:

jsx

```

// src/sections/Hero.jsx
import { useTranslation } from 'react-i18next'
import { useEffect, useRef } from 'react'
import { gsap } from 'gsap'
import fotoYo from '../assets/images/yo.png'

function Hero() {
  const { t } = useTranslation()
  const titleRef = useRef(null)
  const subtitleRef = useRef(null)
  const ctaRef = useRef(null)

  // Animación de entrada con GSAP (reemplaza tu CSS de animación de entrada
  actual)
  useEffect(() => {
    const tl = gsap.timeline()
    tl.from(titleRef.current, { y: 60, opacity: 0, duration: 1, ease:
'power3.out' })
      .from(subtitleRef.current, { y: 40, opacity: 0, duration: 0.8 }, '-
=0.5')
      .from(ctaRef.current, { y: 30, opacity: 0, duration: 0.6 }, '-=0.4')
    }, [])

  return (

```

```

    <section id="inicio" className="hero">
      <div className="hero__content">
        <p className="hero__greeting">{t('hero.greeting')}

```

6.2 about_me.html → src/sections/About.jsx

jsx

```

// src/sections/About.jsx
// Reemplaza: about_me.html + su sección en index.html
import { useTranslation } from 'react-i18next'
import fotoYo from '../assets/images/yo.png'

function About() {
  const { t } = useTranslation()

  return (
    <section id="sobre-mi" className="about">
      <h2 className="section-title">{t('about.title')}

```

```

        <span className="stat__number">3+</span>
        <span className="stat__label">{t('about.yearsExp')}</span>
      </div>
      <div className="stat">
        <span className="stat__number">10+</span>
        <span className="stat__label">{t('about.projectsDone')}</span>
      </div>
    </div>
  </div>
</div>
</section>
)
}

export default About

```

6.3 proyectos.html → src/sections/Projects.jsx + src/components/ProjectCard.jsx

jsx

```

// src/sections/Projects.jsx
// Reemplaza: proyectos.html + js/multilingual-project-modal.js
import { useState } from 'react'
import { useTranslation } from 'react-i18next'
import ProjectCard from '../components/ProjectCard'
import ProjectModal from '../components/ProjectModal'
import { projects } from '../data/projects'

function Projects() {
  const { t, i18n } = useTranslation()
  const [selectedProject, setSelectedProject] = useState(null)
  const lang = i18n.language

  return (
    <section id="proyectos" className="projects">
      <h2 className="section-title">{t('projects.title')}</h2>
      <div className="projects__grid">
        {projects.map(project => (
          <ProjectCard
            key={project.id}
            project={project}
            lang={lang}
            onOpen={() => setSelectedProject(project)}
          />
        ))}
      </div>
    </section>
  )
}

```

```

    )))
  </div>
  { /* Modal reutiliza el proyecto seleccionado */ }
  <ProjectModal
    project={selectedProject}
    onClose={() => setSelectedProject(null)}
  />
</section>
)
}

export default Projects

```

6.4 skills.html → src/sections/Skills.jsx

jsx

```

// src/sections/Skills.jsx
// Reemplaza: skills.html + js/progress-bar.js
import { useTranslation } from 'react-i18next'
import SkillBar from '../components/SkillBar'

// Datos de skills con referencia a tus SVGs actuales
const skillsData = {
  frontend: [
    { name: 'HTML / CSS', percentage: 90, icon:
'/src/assets/icons/web.svg' },
    { name: 'JavaScript', percentage: 80, icon: '/src/assets/icons/coding-
apps.svg' },
    { name: 'React', percentage: 75, icon: '/src/assets/icons/app-
web.svg' },
  ],
  backend: [
    { name: 'Node.js', percentage: 70, icon: '/src/assets/icons/code-
monitor.svg' },
    { name: 'Python', percentage: 60, icon:
'/src/assets/icons/technology.svg' },
  ],
  tools: [
    { name: 'Git / GitHub', percentage: 85, icon: '/src/assets/icons/graph-
arrow.svg' },
    { name: 'Figma', percentage: 65, icon: '/src/assets/icons/ui-
design.svg' },
  ]
}

```

```
function Skills() {
  const { t } = useTranslation()

  return (
    <section id="habilidades" className="skills">
      <h2 className="section-title">{t('skills.title')}</h2>
      <div className="skills__columns">
        {Object.entries(skillsData).map(([category, skills]) => (
          <div key={category} className="skills__group">
            <h3>{t(`skills.${category}`)}</h3>
            {skills.map(skill => (
              <SkillBar key={skill.name} { ... skill } />
            ))}
          </div>
        ))}
      </div>
    </section>
  )
}

export default Skills
```

6.5 cv.html → ruta /cv o descarga directa

Tienes dos opciones:

- **Opción A (recomendada):** Poner el PDF del CV en `public/cv.pdf` y el botón descarga directamente:

jsx

```
<a href="/cv.pdf" download="CV-Alejandro.pdf">Descargar CV</a>
```

- **Opción B:** Crear `src/sections/CV.jsx` con visor PDF embebido (más trabajo, menos impacto).

6.6 header.html → src/components/Navbar.jsx

jsx

```
// src/components/Navbar.jsx
// Reemplaza: header.html + js/jquery.sticky.js + parte de custom.js
import { useState } from 'react'
import { useTranslation } from 'react-i18next'
```

```

import { useScrollPosition } from '../hooks/useScrollPosition'
import { Menu, X } from 'lucide-react'

function Navbar() {
  const { t, i18n } = useTranslation()
  const scrollY = useScrollPosition()
  const [menuOpen, setMenuOpen] = useState(false)
  const isScrolled = scrollY > 50

  const links = [
    { href: '#inicio',      key: 'nav.home' },
    { href: '#sobre-mi',    key: 'nav.about' },
    { href: '#proyectos',   key: 'nav.projects' },
    { href: '#habilidades', key: 'nav.skills' },
    { href: '#contacto',    key: 'nav.contact' },
  ]

  return (
    <nav className={`navbar ${isScrolled ? 'navbar--scrolled' : ''}`}>
      <a href="#inicio" className="navbar__logo">
        Alejandro<span>.</span>
      </a>

      <ul className={`navbar__links ${menuOpen ? 'navbar__links--open' : ''}`}>
        {links.map(link => (
          <li key={link.key}>
            <a href={link.href} onClick={() => setMenuOpen(false)}>
              {t(link.key)}
            </a>
          </li>
        ))}
      </ul>

      <div className="navbar__actions">
        {/* Selector de idioma */}
        <button
          className="lang-toggle"
          onClick={() => i18n.changeLanguage(i18n.language === 'es' ? 'en' :
'es')}
        >
          {i18n.language === 'es' ? 'EN' : 'ES'}
        </button>

        {/* Menú hamburguesa (mobile) */}
        <button

```

```

      className="navbar__hamburger"
      onClick={() => setMenuOpen(!menuOpen)}
      aria-label="Menú"
    >
      {menuOpen ? <X size={24} /> : <Menu size={24} />}
    </button>
  </div>
</nav>
)
}

export default Navbar

```

6.7 404.html → manejado por React Router

Ya cubierto en el punto 5.1 con `NotFound.jsx`.

6.8 Carpeta nube/ → eliminada

`nube.html`, `nube.css`, `nube.js` y `rain.js` son experimentos o efectos que se reemplazan por la escena `Three.js`. Si el efecto de lluvia te gustaba específicamente, se puede recrear con partículas en `Three.js` dentro de `SpaceBackground.jsx`.

FASE 7 — Implementar la escena 3D del hero

7.1 src/3d/SpaceBackground.jsx — fondo estrellado completo

- ☐ Crear el componente con `Canvas` de R3F
- ☐ Añadir `<Stars>` de `dreij`
- ☐ Añadir `<EffectComposer>` + `<Bloom>` de `postprocessing`
- ☐ Montar en `App.jsx` como primer hijo con `className="canvas-wrapper"`

7.2 src/3d/FloatingPlanet.jsx — planeta decorativo en el hero

- ☐ Crear una esfera con `<mesh>` y `<sphereGeometry>`
- ☐ Aplicar textura o material `MeshStandardMaterial` con color azul/violeta
- ☐ Usar `<Float>` de `dreij` para que flote suavemente
- ☐ Añadir `<ambientLight>` y `<pointLight>` para iluminación

7.3 Integrar el canvas en el hero

- ☐ El `<Canvas>` 3D va en `position: fixed` con `z-index: 0`

- ☐ El contenido HTML del hero va en `position: relative` con `z-index: 1`
-

FASE 8 — Animaciones con GSAP

8.1 Instalar el plugin de ScrollTrigger

bash

```
# Ya incluido con gsap, solo registrar el plugin:
```

jsx

```
// En App.jsx o en cada sección que lo use:
import { gsap } from 'gsap'
import { ScrollTrigger } from 'gsap/ScrollTrigger'
gsap.registerPlugin(ScrollTrigger)
```

8.2 Animaciones por sección

- ☐ **Hero:** entrada con `gsap.timeline()` (nombre, rol, botones aparecen en cascada)
- ☐ **About:** fade-in al entrar en viewport con `ScrollTrigger`
- ☐ **Projects:** cards aparecen en stagger (una tras otra) al hacer scroll
- ☐ **Skills:** barras animadas ya cubiertas con `IntersectionObserver` en `SkillBar`
- ☐ **Contact:** fade-in del formulario al entrar en viewport

8.3 Scroll suave con Lenis

jsx

```
// src/hooks/useLenis.js
import { useEffect } from 'react'
import Lenis from 'lenis'
import { ScrollTrigger } from 'gsap/ScrollTrigger'

export function useLenis() {
  useEffect(() => {
    const lenis = new Lenis()
    // Sincroniza Lenis con GSAP ScrollTrigger
    lenis.on('scroll', ScrollTrigger.update)

    const animate = (time) => {
```

```

    lenis.raf(time)
    requestAnimationFrame(animate)
  }
  requestAnimationFrame(animate)

  return () => lenis.destroy()
}, [])
}

```

Úsalo en `App.jsx`:

jsx

```

import { useLenis } from './hooks/useLenis'
function App() {
  useLenis()
  // ...
}

```

FASE 9 — Ensamblaje final en `App.jsx`

jsx

```

// src/App.jsx – versión final completa
import { Routes, Route } from 'react-router-dom'
import './src/locales/index.js' // Inicializa i18next
import { useLenis } from './hooks/useLenis'
import SpaceBackground from './3d/SpaceBackground'
import Navbar from './components/Navbar'
import Hero from './sections/Hero'
import About from './sections/About'
import Projects from './sections/Projects'
import Skills from './sections/Skills'
import Contact from './sections/Contact'
import NotFound from './sections/NotFound'

function HomePage() {
  return (
    <>
      <Hero />
      <About />
      <Projects />
      <Skills />
    </>
  )
}

```

```

        <Contact />
      </>
    )
  }

function App() {
  useLenis()

  return (
    <>
      { /* Fondo 3D – fijo, detrás de todo el contenido */ }
      <SpaceBackground />

      { /* Navbar – siempre visible */ }
      <Navbar />

      { /* Contenido principal sobre el canvas */ }
      <div className="content-layer">
        <Routes>
          <Route path="/" element={<HomePage />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </div>
    </>
  )
}

export default App

```

FASE 10 — Archivos que se eliminan definitivamente

Una vez completada la migración, estos archivos ya no tienen equivalente porque su funcionalidad fue absorbida por React:

Archivo original	Por qué se elimina
js/jquery.min.js	React no necesita jQuery
js/jquery.sticky.js	useScrollPosition hook
js/jquery.magnific-popup.min.js	ProjectModal.jsx nativo
js/magnific-popup-options.js	Integrado en ProjectModal.jsx
js/lenguaje.js	i18next

Archivo original	Por qué se elimina
js/multilingual-project-modal.js	Integrado en ProjectModal.jsx
js/slide.js	Diseño sin carrusel
css/magnific-popup.css	CSS del modal va en el componente
css/slide.css	Diseño sin carrusel
css/bootstrap-icons.css	lucide-react
fonts/bootstrap-icons.woff	lucide-react
fonts/bootstrap-icons.woff2	lucide-react
nube/nube.html	Reemplazado por Three.js
nube/nube.css	Reemplazado por Three.js
nube/nube.js	Reemplazado por Three.js
nube/rain.js	Reemplazado por Three.js
images/mundo.gif	Reemplazado por escena 3D

FASE 11 — Deploy

11.1 Preparar el build de producción

- ☐ Ejecutar `npm run build` — genera la carpeta `dist/`
- ☐ Verificar con `npm run preview` que todo funciona en modo producción

11.2 Deploy en GitHub Pages (misma plataforma que tienes ahora)

- ☐ Instalar el plugin: `npm install --save-dev gh-pages`
- ☐ En `vite.config.js`, añadir el base del repositorio:

js

```
export default {
  base: '/nombre-de-tu-repo/',
}
```

- ☐ En `package.json`, añadir el script de deploy:

json

```
"scripts": {  
  "deploy": "npm run build && gh-pages -d dist"  
}
```

- ☐ Ejecutar `npm run deploy`
- ☐ Tu `CNAME` actual: copiarlo a la carpeta `public/` para que Vite lo incluya en el build

Resumen de progreso por fases

Fase	Descripción	Estado
0	Preparar entorno y backup	[]
1	Crear proyecto React + Vite	[]
2	Migrar imágenes e íconos	[]
3	Configuración global (index.html, globals.css)	[]
4	Internacionalización (reemplazar <code>lang/</code> + <code>lenguaje.js</code>)	[]
5	Migrar archivos JS a componentes y hooks	[]
6	Migrar vistas HTML a secciones React	[]
7	Implementar escena 3D con Three.js	[]
8	Animaciones con GSAP + scroll suave con Lenis	[]
9	Ensamblaje final en App.jsx	[]
10	Limpieza de archivos obsoletos	[]
11	Build y deploy	[]